Job Scheduler Platform

A production-inspired distributed job scheduling system
Multi-tenant queues · Atomic job claiming · Retry & DLQ · Live dashboard

Repository: https://github.com/Sk0108/Job-Scheduler

Stack: Node.js / Express / Prisma / PostgreSQL / Redis / React / Socket.IO

Name: [Your Name]
Student / Roll No.: [Your ID]
Course / Subject: [Course Name]
Date: [Submission Date]

Table of Contents

1. Objective & Overview

This project is a full-stack, production-inspired distributed job scheduling platform. It supports multi-tenant organizations, projects, and job queues; five job creation modes (immediate, delayed, scheduled, recurring/cron, and batch); a horizontally-scalable worker fleet that atomically claims and executes jobs concurrently; configurable retry strategies with a dead-letter queue; and a live operational dashboard with drag-and-drop queue management, a calendar view, and real-time updates.

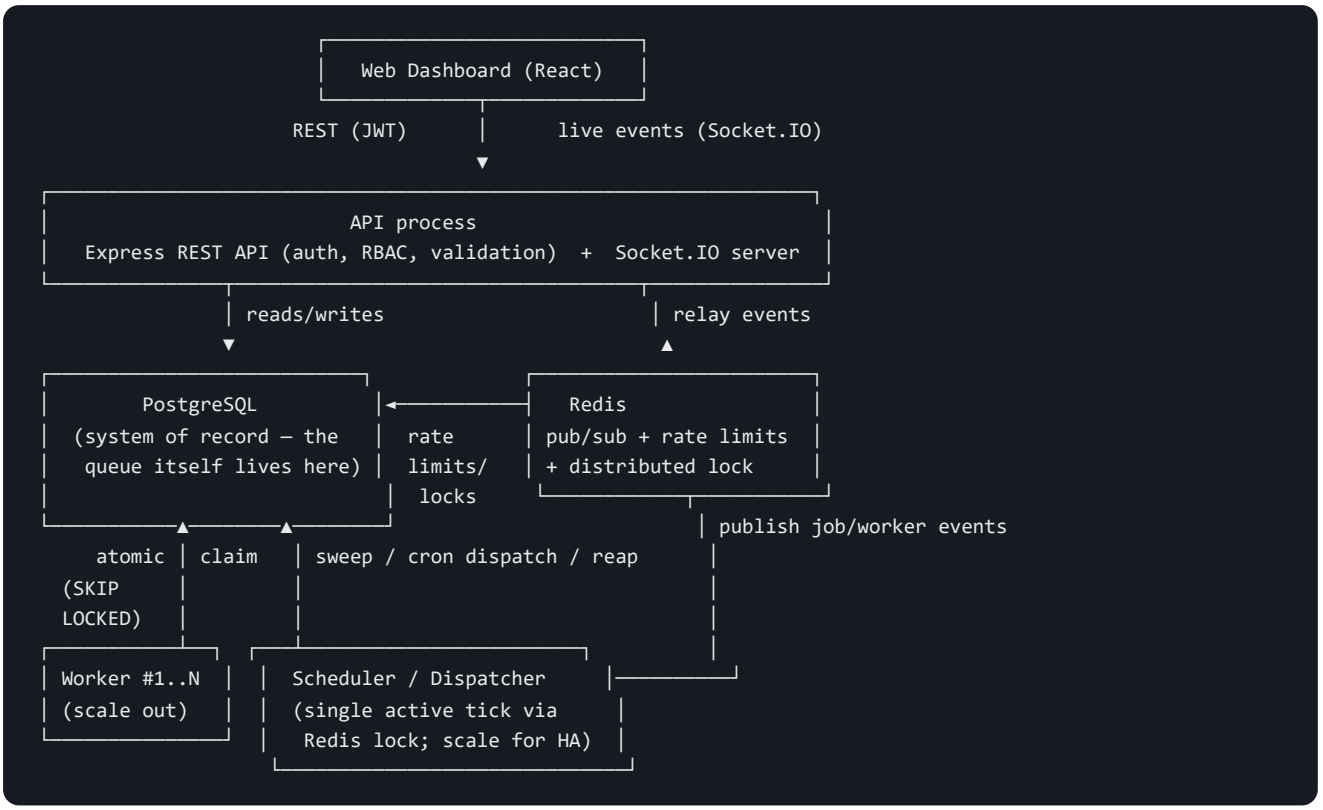
The system is built as an npm-workspaces monorepo of five backend packages (`db` , `shared` , `core` , `api` , `worker` , `scheduler`) plus a React dashboard (`apps/web`), so that the atomic-claim/retry/DLQ logic that all three runtime processes depend on lives in exactly one place (`@jsp/core`) rather than being duplicated.

Core requirements delivered

- Authentication (JWT access + rotating refresh tokens) and organization/project management with role-based access control (OWNER/ADMIN/MEMBER/VIEWER).
- Queue configuration: priority, concurrency limits, distributed rate limiting, retry policy, pause/resume, live stats.
- Immediate, delayed, scheduled, recurring (cron), and batch job creation via REST.
- Worker service: polls queues, atomically claims jobs (no duplicate execution under concurrency), executes concurrently, sends heartbeats, supports graceful shutdown.
- Full lifecycle: Queued → Scheduled → Claimed → Running → Completed, with retries and a Dead Letter Queue for permanent failures.
- Configurable retry strategies: fixed delay, linear backoff, exponential backoff (with jitter).
- Execution logs, retry history, worker assignment, timestamps, and metrics for every job.
- Web dashboard: queue management, job explorer, execution logs, worker monitor, drag-and-drop board, calendar, throughput/priority charts, dark/light themes, first-login onboarding tour.

2. System Architecture

Process diagram

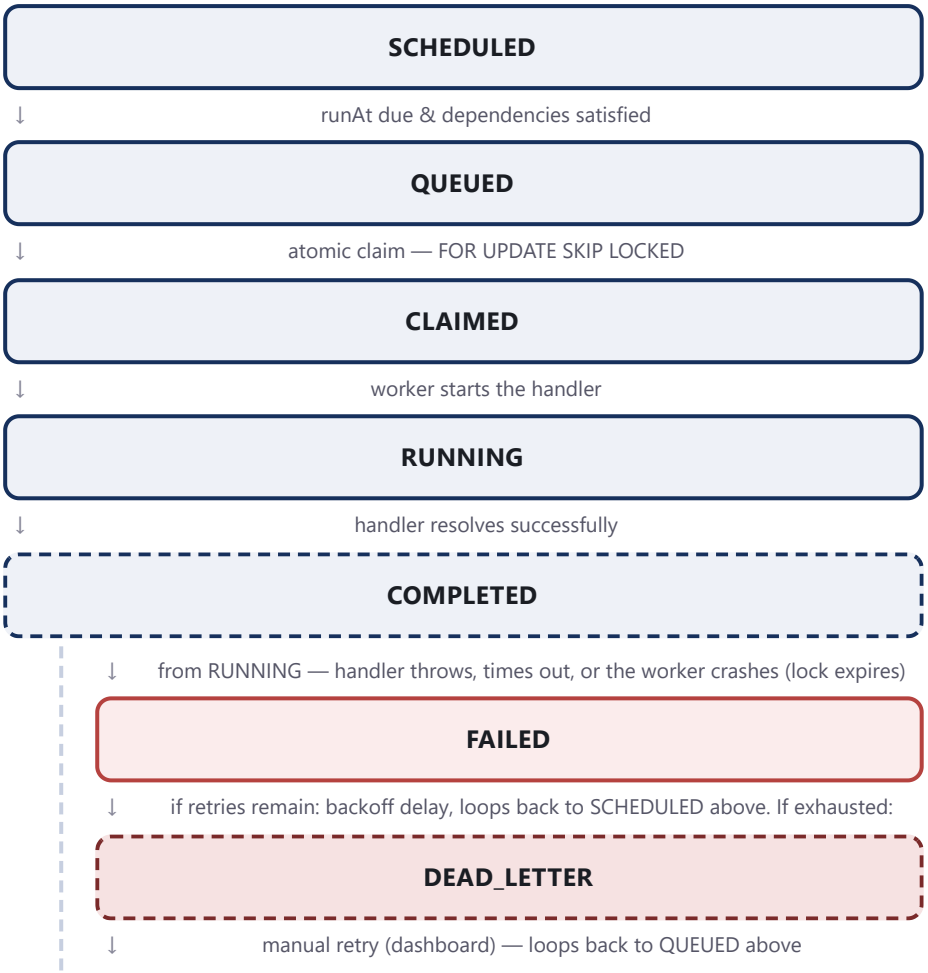


Process	Responsibility	Scales by
API	REST endpoints, JWT auth, RBAC, validation, Socket.IO gateway	Stateless — run N behind a load balancer
Worker	Polls queues, atomically claims jobs, executes concurrently, heartbeats, graceful shutdown	Stateless — run N; claiming is safe under concurrency
Scheduler	Promotes due SCHEDULED jobs, dispatches cron JobDefinitions, reaps stale claims (crash recovery)	Run N for HA — a Redis lock ensures only one replica does work per tick
PostgreSQL	Single system of record for every entity — no dual-write between "queue state" and "job state"	Vertical, or read replicas for dashboard queries
Redis	Cross-process event bus, distributed rate limiting, distributed lock	Managed Redis / cluster

Why Postgres as the queue, not a message broker

Jobs live in a Postgres table; atomic dequeue uses `FOR UPDATE SKIP LOCKED` inside a single `UPDATE ... FROM (SELECT ... FOR UPDATE SKIP LOCKED)` statement. Job state (status, attempt count, retry history, DLQ) and queue state (what's claimable right now) are the same data — splitting them across a broker and a database would mean either a dual-write that can disagree, or losing SQL's ability to query job history directly. A single ACID store removes an entire class of consistency bugs, and `SKIP LOCKED` claim throughput is well past what an internal scheduler needs at this scale.

Job lifecycle (state machine)



Every transition above is implemented in exactly one place (`packages/core/src/lifecycle.ts` and `claim.ts`) and called from whichever process triggers it — worker, scheduler reaper, or API — so the retry-vs-dead-letter decision can never disagree between call sites.

Live updates: polling + WebSocket

The dashboard polls every 5s via TanStack Query as a correctness baseline, and layers Socket.IO push on top: the worker/scheduler publish lifecycle events to a Redis channel; the API subscribes and relays them into per-project Socket.IO rooms; the dashboard invalidates the relevant caches instantly on receipt. If the socket drops, the 5s poll still keeps the UI correct — the socket is a latency optimization, not a correctness dependency.

3. Database Design

13 tables, fully normalized (3NF) with one deliberate, documented denormalization (per-job retry overrides — see §10). Every scalar field is explicitly `@map`'d to a conventional snake_case Postgres column, since the atomic-claim and metrics queries run as raw SQL for capabilities the query builder can't express (`FOR UPDATE SKIP LOCKED`, `date_trunc` bucketing).

Identity & Access

User — id (PK), email (unique), password_hash, is_active
RefreshToken — user_id (FK→User, cascade), token_hash (unique, hashed not raw)
Organization — id (PK), slug (unique)
OrganizationMember — org_id+user_id (FK, cascade), role enum, unique(org,user)

Projects, Queues, Policies

Project — org_id (FK, cascade), unique(org,slug)
ApiKey — project_id (FK, cascade), key_hash (unique)
RetryPolicy — project_id (FK, cascade), strategy/maxRetries/baseDelay/maxDelay, unique(project,name)
Queue — project_id (FK, cascade), default_retry_policy_id (FK→RetryPolicy, SET NULL), unique(project,slug)

Jobs & Lifecycle

Job — queue_id (FK, cascade), batch_id/job_definition_id/claimed_by_worker_id (FK, SET NULL), unique(queue_id, idempotency_key)
JobDependency — self-join on Job (job_id, depends_on_job_id), both cascade
JobDefinition (cron) — queue_id (FK, cascade), next_run_at indexed
Batch — project_id + queue_id (FK, cascade)

Execution & Workers

JobExecution — job_id (FK, cascade), worker_id (FK, SET NULL) — one row per attempt, audit trail
JobLog — job_id (FK, cascade), execution_id (FK, cascade)
DeadLetterEntry — job_id (FK, cascade), payload_snapshot (jsonb)
Worker / **WorkerHeartbeat** — worker_id (FK, cascade)

Indexing strategy (chosen for actual hot-path queries, not speculative)

Index	Serves
`Job(queueId, status, priority, runAt)`	The atomic claim query's exact filter/sort — the single hottest query in the system
`Job(status, runAt)`	Scheduler's system-wide sweep for due SCHEDULED jobs
`JobDefinition(nextRunAt)`	Cron dispatcher's "what's due" query
`JobLog(jobId, timestamp)`, `JobExecution(jobId)`	Job detail page's execution/log history
Every FK column	Cascading deletes and parent-scoped dashboard lookups avoid table scans

Cascading behavior

Cascade delete: Organization→Project→Queue→Job and Job→(Execution/Log/DLQ) — deleting a project is meant to delete everything it owns. **SET NULL:** Queue.defaultRetryPolicyId, Job.claimedByWorkerId, Job.createdById, Job.batchId — the referenced row (a retired worker, a deleted policy) can disappear without invalidating the job's own history.

Normalization trade-off

Job carries optional `maxRetries` / `retryStrategy` / `baseDelayMs` / `maxDelayMs` columns that, when NULL, fall back to the queue's `defaultRetryPolicy`. Modeling this "purely" would force every job to point at a RetryPolicy row even for a

one-off override. The override columns keep the common case (inherit the queue's policy) fully normalized while making the rare per-job override cheap — see `resolveRetryPolicy()` in `packages/core/src/lifecycle.ts`.

Performance

Job/execution/log payloads use `jsonb` (arbitrary, application-defined, never queried by internal field — EAV normalization would be strictly worse). `JobExecution` is kept separate from `Job` specifically so retry history is queryable without ever mutating past attempts.

4. Backend Engineering

Layered architecture

`packages/api` (Express routes/middleware) → `packages/core` (reliability engine: claim, lifecycle, dispatch, stats, lock, events) → `packages/db` (Prisma client) → PostgreSQL. The worker and scheduler processes call the same `@jsp/core` functions the API's manual "retry" button calls, so there is exactly one implementation of "what happens when a job fails" in the whole system.

Atomic job claiming — the core reliability guarantee

```
WITH candidate AS (
  SELECT id FROM jobs
  WHERE queue_id = $1 AND status = 'QUEUED' AND run_at <= now()
  ORDER BY priority DESC, run_at ASC
  LIMIT $2
  FOR UPDATE SKIP LOCKED
)
UPDATE jobs
SET status = 'CLAIMED', claimed_by_worker_id = $3,
    claimed_at = now(), lock_expires_at = now() + interval '...'
FROM candidate WHERE jobs.id = candidate.id
RETURNING jobs.*;
```

Implemented as one statement so there is no window between "pick a row" and "mark it claimed" for a second worker to race into. Verified under real concurrency with an integration test that runs 5 simulated workers against the same 40-job queue and asserts every job is claimed exactly once (see §9 Testing).

Validation, auth, error handling, logging

- **Validation:** every request body/query validated with Zod schemas before reaching a handler.
- **Auth:** JWT access tokens (15m) + rotating refresh tokens (7d, revoked-on-use, stored only as a SHA-256 hash).
- **RBAC:** org-level roles (OWNER>ADMIN>MEMBER>VIEWER) resolved for every nested resource (queue→project→org, job→queue→project→org).
- **Error handling:** a single Express error middleware maps ApiError / ZodError / Prisma known-request errors to a consistent `{ error: { code, message, details } }` shape and status code.
- **Logging:** structured request/response logging via pino-http; unhandled errors logged with full stack server-side.
- **Idempotent job creation:** unique constraint on (queueId, idempotencyKey); a duplicate submission returns the existing job (200) instead of racing a check-then-insert.

Package layout

Package	Contents
`@jsp/db`	Prisma schema, migrations, seed script
`@jsp/shared`	Pure logic: retry backoff math, cron helpers, pagination, failure-summary heuristic
`@jsp/core`	Atomic claim, retry/DLQ lifecycle, dispatch, stats, distributed lock, Redis event bus
`@jsp/api`	Express REST API + Socket.IO live updates
`@jsp/worker`	Poll → claim → execute → heartbeat → graceful shutdown
`@jsp/scheduler`	Scheduled→queued sweep, cron dispatch, stale-lock reaper

`apps/web`	React dashboard

5. Reliability & Concurrency

Mechanisms

Mechanism	How
Atomic claim	Single `FOR UPDATE SKIP LOCKED` statement — two workers can never claim the same job; many workers fan out across distinct rows instead of blocking each other.
Crash recovery	Every claim carries a `lockExpiresAt`, extended to cover the job's timeout when execution starts. If a worker dies mid-job, the scheduler's reaper notices the expired lock and routes it through the normal retry/DLQ decision — no job is silently stuck.
Distributed lock	Scheduler wraps each tick in a Redis `SET NX PX` mutex so running two scheduler replicas for HA can't double-dispatch the same cron job.
Distributed rate limiting	A queue's `rateLimitPerSecond` is enforced fleet-wide via a Redis counter keyed by second, not per-worker — the limit holds regardless of worker count.
Graceful shutdown	On SIGTERM/SIGINT the worker stops claiming new jobs, drains in-flight jobs (bounded grace period), marks itself OFFLINE, then exits.
Configurable retries	FIXED / LINEAR / EXPONENTIAL backoff, each with optional ±25% jitter to avoid thundering-herd retries.
Concurrency limits	Per-queue `concurrencyLimit` and per-worker semaphore both enforced before a claim is attempted.

Retry backoff formulas

```
FIXED:        delay = baseDelayMs
LINEAR:       delay = baseDelayMs * attempt
EXPONENTIAL:  delay = baseDelayMs * 2^(attempt - 1)
              delay = min(delay, maxDelayMs)
              if jitter: delay *= (0.75 + random() * 0.5)   // full ±25% jitter
```

Workflow dependencies (bonus)

A job created with `dependsOn` stays SCHEDULED until every prerequisite reaches COMPLETED, enforced by a `NOT EXISTS` clause in the scheduler's due-job sweep — a downstream job can never start before its inputs are ready.

6. API Design

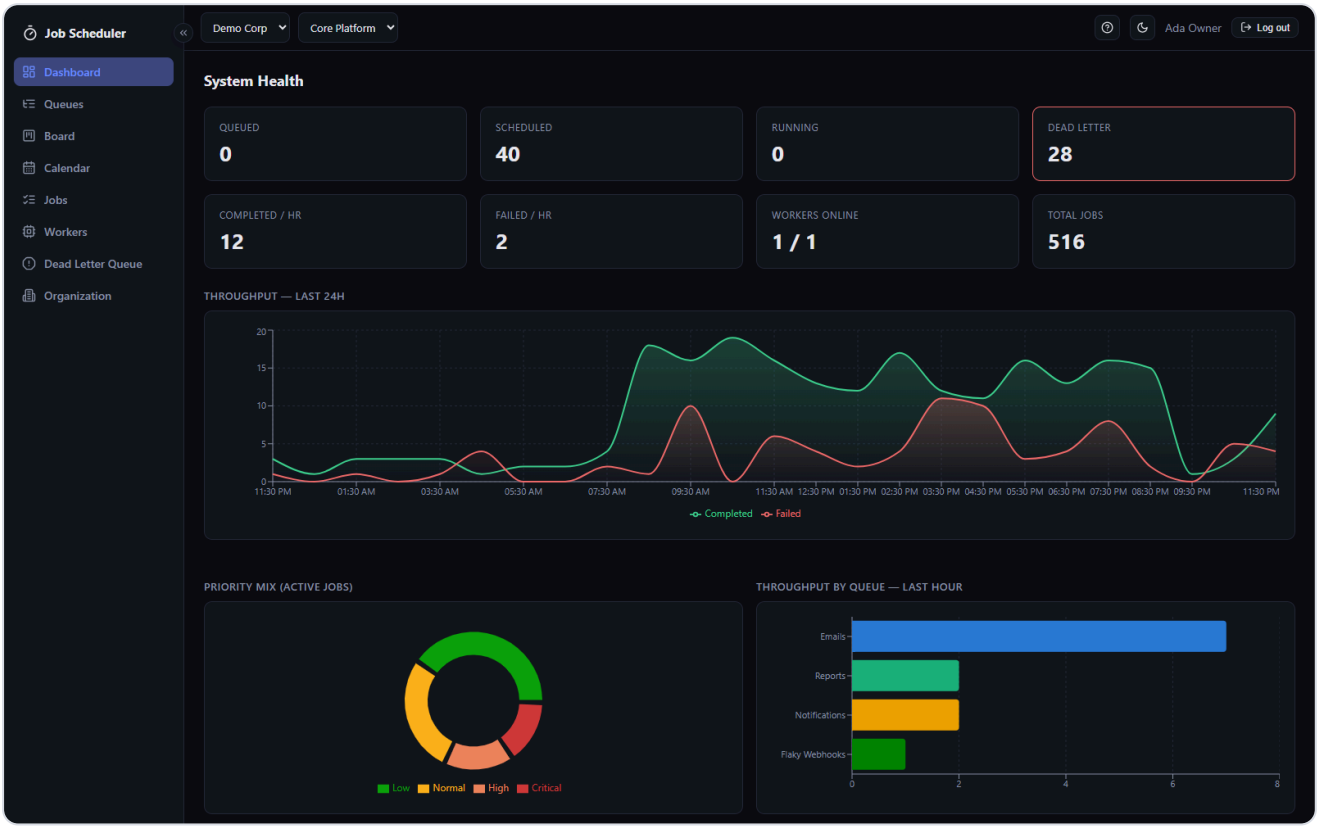
Base URL `/api/v1`. Consistent conventions across every endpoint: cursor-free page/pageSize pagination (`{ data, page, pageSize, total, totalPages }`), comma-separated multi-value filters (`status=FAILED,DEAD_LETTER`), a single error shape, and role minimums enforced per-endpoint.

Resource	Key endpoints
Auth	register / login / refresh (rotating) / logout / me
Organizations	CRUD, members with role management
Projects	CRUD, retry policies, system health
Queues	CRUD, pause/resume, stats (status counts, throughput, duration histogram)
Jobs	create (immediate/delayed/scheduled/batch/dependsOn), list+filter+paginate, detail (executions/logs/failure summary), cancel, retry, move (drag-and-drop), calendar range
Job Definitions	cron CRUD, pause/resume
Dead Letter Queue	list, retry, resolve
Workers	fleet list/detail with heartbeat history
Metrics	health, throughput (hourly buckets), per-queue stats, priority distribution

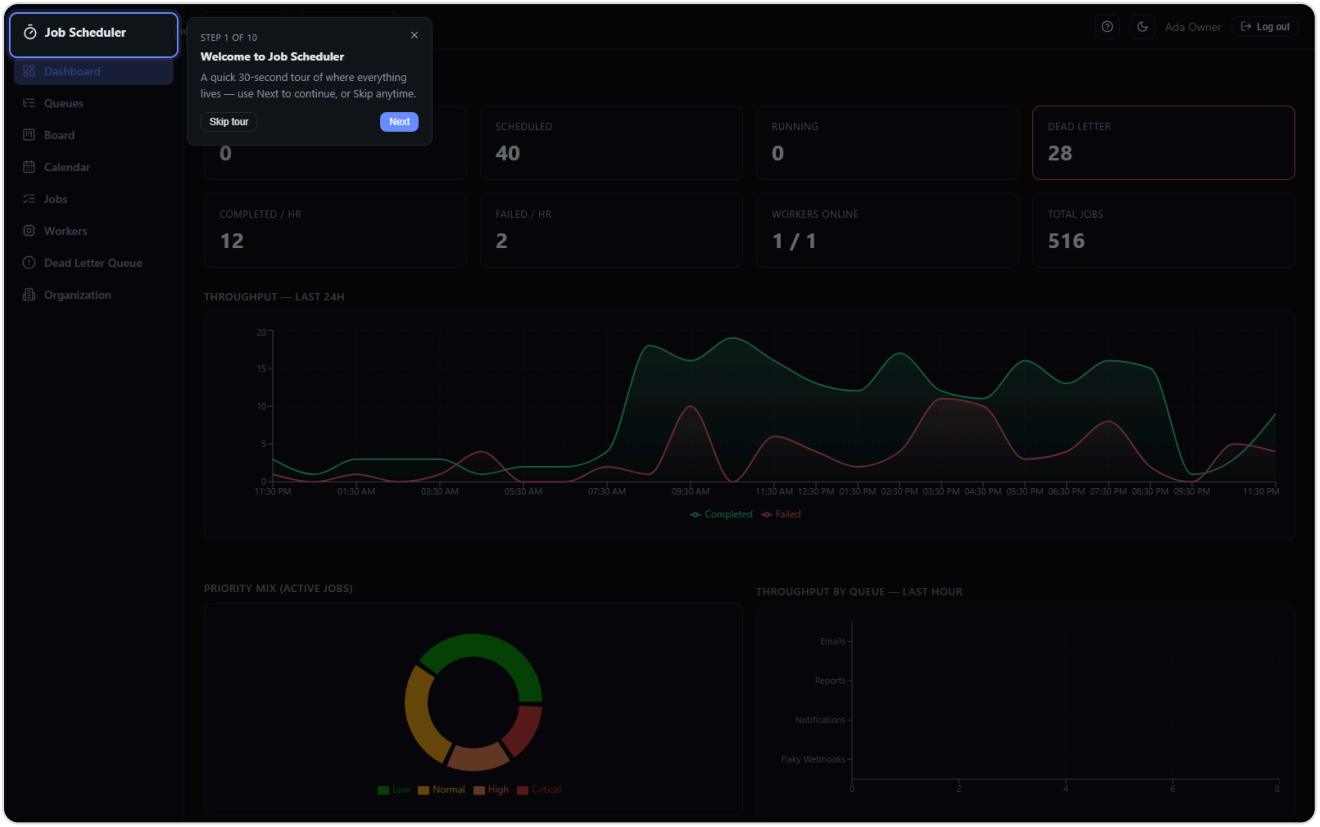
Full endpoint-by-endpoint documentation with request/response examples: `docs/api.md` in the repository.

7. Frontend & UX

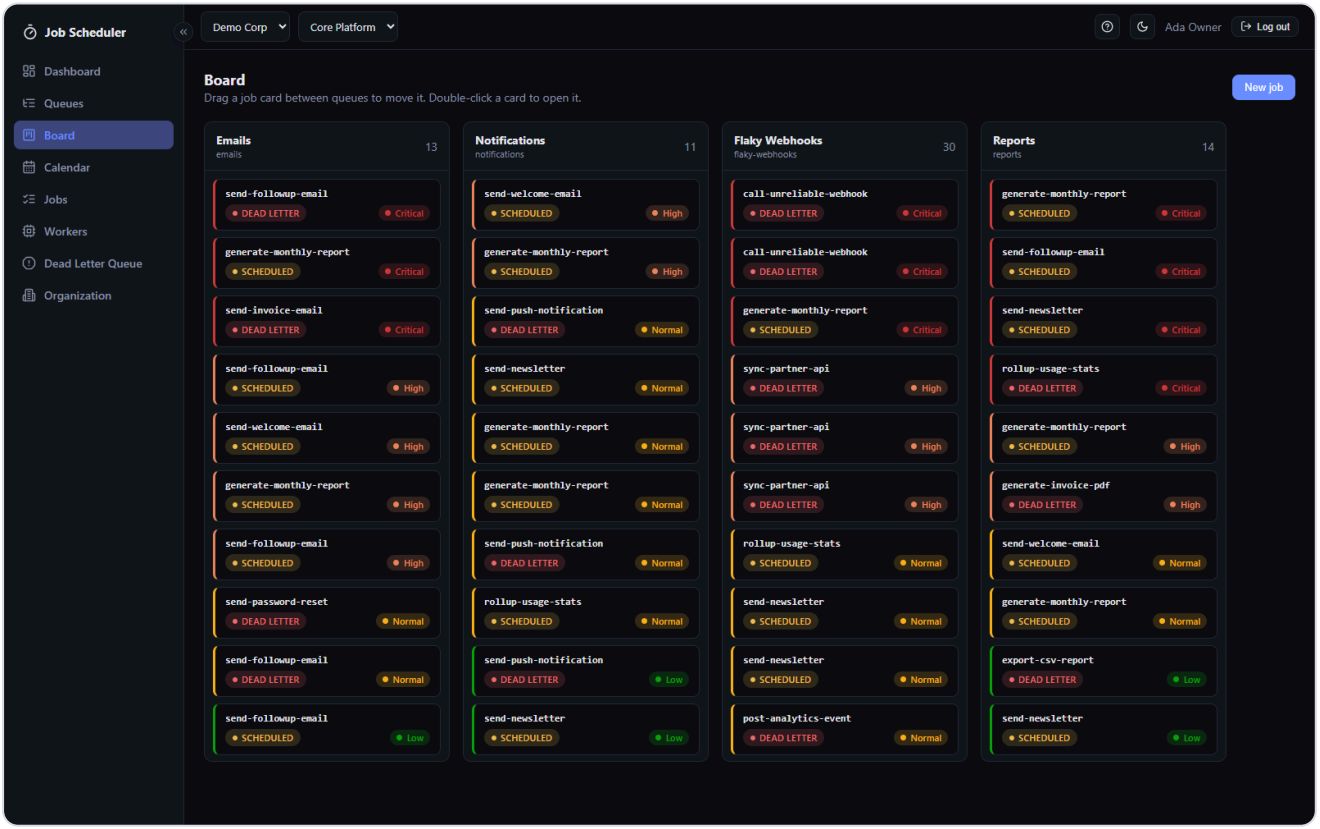
React + TanStack Query + Recharts + Socket.IO client + framer-motion + @dnd-kit. All screenshots below are captured from the actual running application against a seeded, realistic dataset (450+ historical jobs spread across 48 hours, four queues, all priority bands, genuine failures and dead-letters).



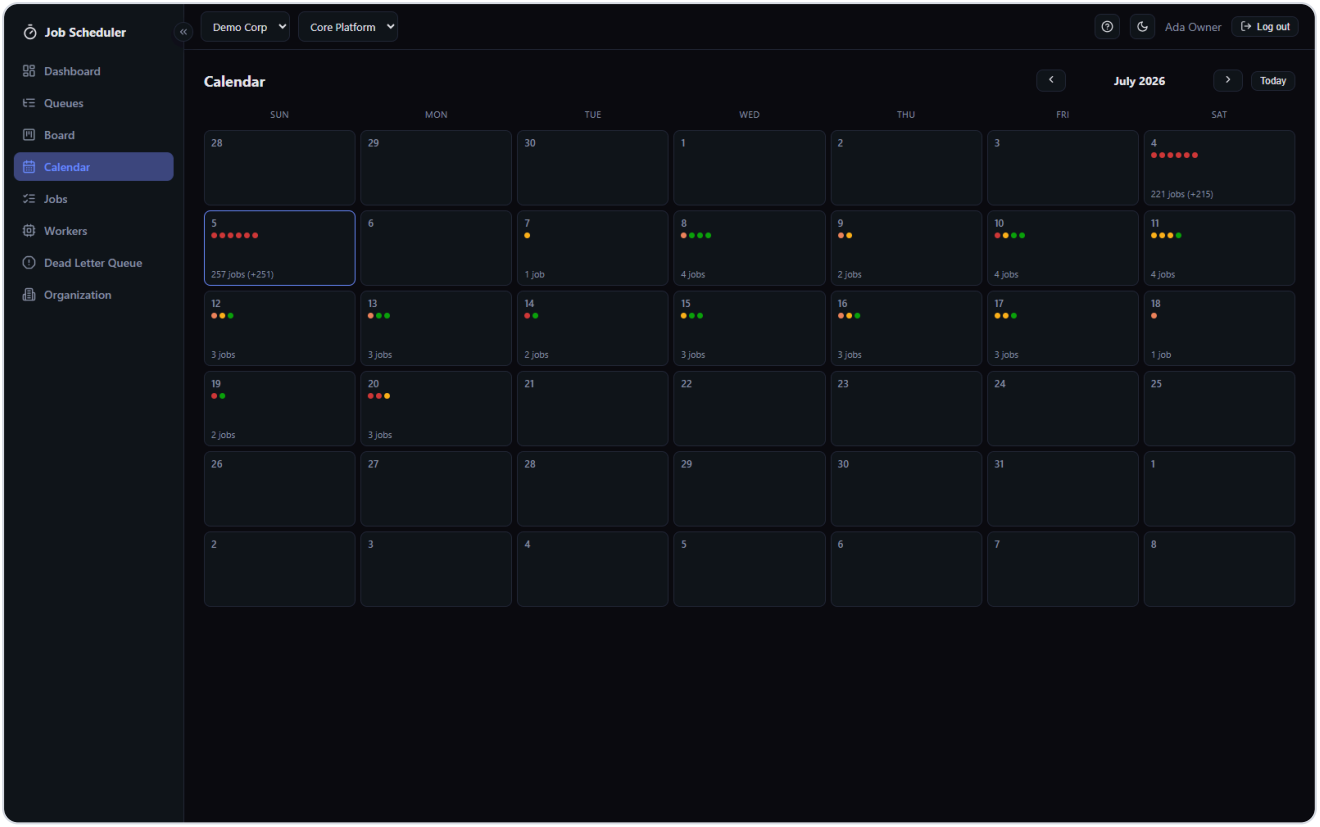
Dashboard — animated stat tiles, 24h throughput (completed vs. failed), priority-mix donut, per-queue throughput comparison. Collapsible sidebar with an always-visible edge toggle.



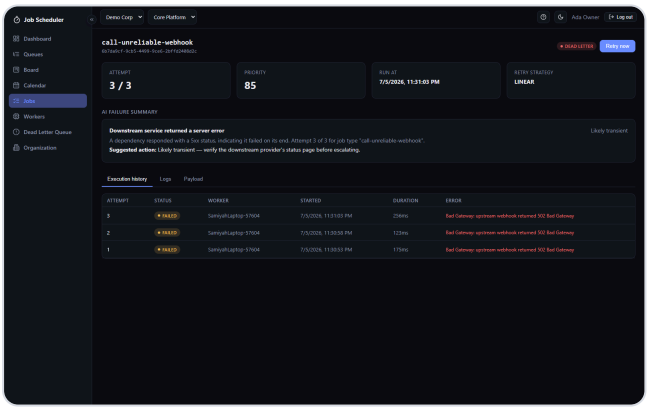
First-login onboarding tour — spotlight + callout bubble walks a new user through every nav item and control; replayable anytime via the help icon.



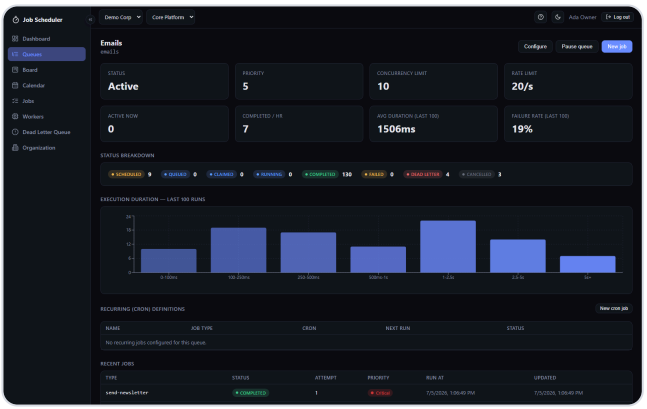
Board — drag-and-drop kanban, one column per queue. Dragging a card between columns calls `POST /jobs/:id/move`. Cards are color-coded by priority band (Low/Normal/High/Critical).



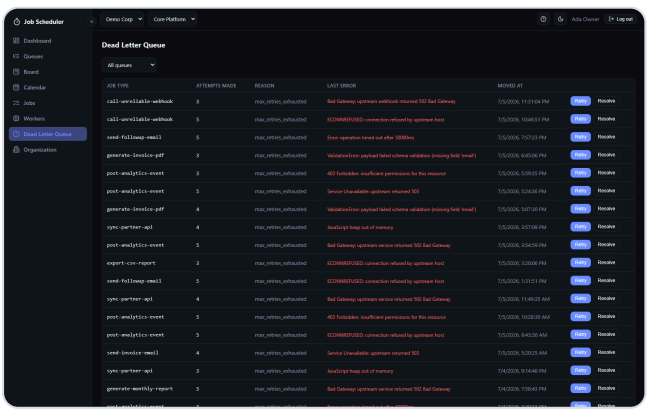
Calendar — jobs by day, color-coded dots by priority; clicking a day lists that day's jobs grouped by priority band.

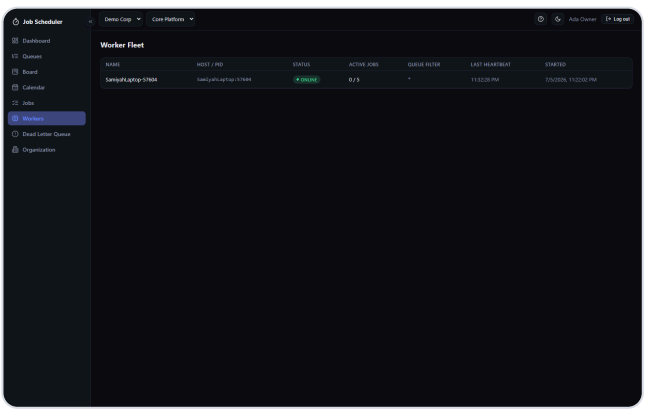


Job detail — execution history, retry attempts, and a rule-based AI failure-summary classification (with an explicit swap-in seam for a real LLM call).



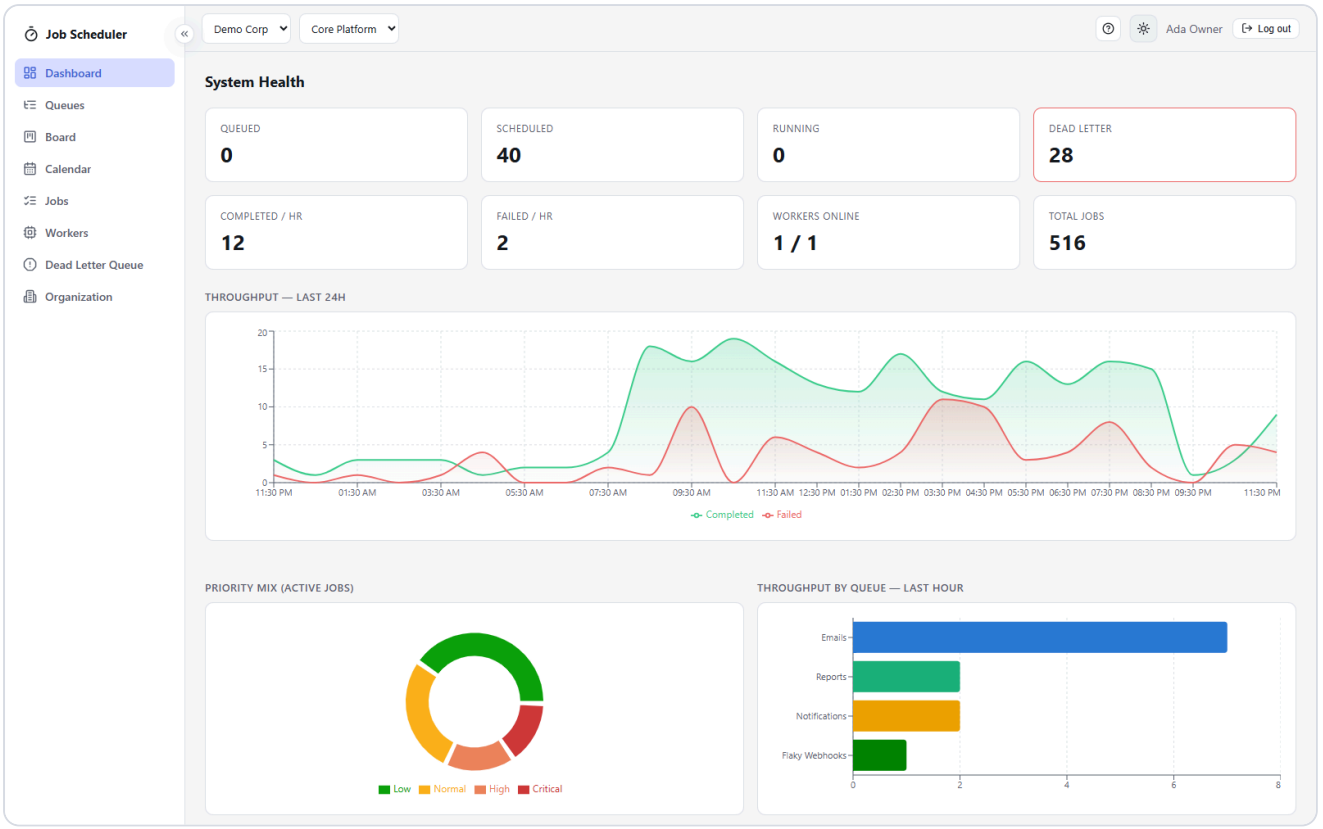
Queue detail — status breakdown, execution-duration histogram, cron definitions, live config editing.





Dead Letter Queue — retry or resolve permanently-failed jobs.

Worker fleet monitor — status, heartbeat freshness, active job count.



Light mode — explicit theme toggle (not just OS-preference-driven), persisted per device, applied before first paint to avoid a flash of the wrong theme.

UX highlights beyond the core requirement

- Live updates via Socket.IO with toast notifications on key events (job dead-lettered, worker offline, job moved), backstopped by 5s polling.
- Framer-motion animations throughout: page transitions, animated count-up stat tiles, staggered table reveals, live-status pulsing badges.
- Priority color-coding reuses a single fixed severity ramp (good→warning→serious→critical) everywhere — job table, board cards, calendar dots — always paired with a text label, never color alone.

8. Documentation

Document	Contents
`README.md`	Setup instructions, stack overview, monorepo layout, scripts reference
`docs/architecture.md`	Process diagram, job lifecycle state machine, reliability mechanisms
`docs/er-diagram.md`	Full ER diagram, PK/FK/index/normalization/cascade rationale
`docs/api.md`	Endpoint-by-endpoint REST reference with request/response examples
`docs/design-decisions.md`	Every major trade-off: Postgres-as-queue, retry/jitter formula, RBAC model, worker visibility, AI failure summaries, known limitations

This PDF itself is submitted alongside the repository as a consolidated overview mapped to the evaluation rubric.

9. Testing

Automated tests run against a **real PostgreSQL instance**, not mocks — two real bugs during development (an unmapped column name, and a `::uuid` cast against a `text` column) were caught exactly because a mocked Prisma client would have "succeeded" against SQL that real Postgres rejects. Integration suites auto-skip if `DATABASE_URL` is unreachable, so the full suite is safe to run anywhere.

Suite	Covers
`packages/shared`	Retry backoff math (fixed/linear/exponential + jitter bounds), failure-summary classification
`packages/core` (unit)	Retry-policy precedence (job override → queue policy → system default)
`packages/core` (integration)	**Atomic claim under real concurrency** — 5 simulated workers race for 40 jobs; asserts every job claimed exactly once, no duplicates, respects claim limits and future `runAt`. Retry-vs-dead-letter decision correctness.
`packages/api` (integration)	Full auth flow incl. refresh-token rotation/replay rejection, RBAC enforcement (VIEWER blocked from admin actions), idempotent job creation, delayed vs. immediate job status
`packages/worker`	Concurrency semaphore never exceeds capacity; drain-on-shutdown behavior

Run with `npm test` from the repository root.

10. Key Design Decisions & Trade-offs

Full detail in `docs/design-decisions.md`; summary of the most consequential ones:

- **Postgres as the queue** instead of a dedicated broker — one consistent system of record instead of a broker/DB dual-write that can disagree.
- **Raw SQL for claim/dispatch/metrics**, with the schema's columns explicitly `@map`'d to snake_case so hand-written SQL matches Postgres convention.
- **Redis pub/sub as the event bus** between background processes and the dashboard — the worker/scheduler never need to accept inbound connections themselves.
- **Distributed lock for the scheduler** — makes running multiple scheduler replicas for HA safe by construction, not by operational policy.
- **Org-level RBAC**, not per-project roles — matches the brief's actual requirement with one join instead of two; extensible later via an optional ProjectMember override table.
- **Job-level retry overrides** instead of mandatory per-job policies — keeps the common case normalized while making rare overrides cheap.
- **Rule-based AI failure summaries** with an explicit LLM seam — deterministic and free to run on every failure; the function's input/output contract is intentionally LLM-shaped so swapping in a real model call is a one-function change.
- **Known limitation:** a worker killed without SIGTERM leaves a stale "ONLINE" row (mitigated via heartbeat-age staleness detection in the UI, not auto-reaped, to preserve history).

11. Setup Instructions

```
npm install
cp .env.example .env
docker compose up -d              # Postgres + Redis
npm run db:migrate
npm run db:seed                   # demo org/project/queues/jobs

npm run dev:api                   # http://localhost:4000
npm run dev:worker
npm run dev:scheduler

cp apps/web/.env.example apps/web/.env
npm run dev:web                   # http://localhost:5174

npm test                          # full test suite
```

Demo login: `admin@demo.io` (OWNER) or `member@demo.io` (MEMBER), password `Password123!`, created by the seed script.

Full details, including production build/deploy notes, are in the repository's `README.md`.

12. Deliverables & Repository

Source code (public repository):
https://github.com/Sk0108/Job-Scheduler

Deliverable	Location
Source code with setup instructions	Repository root `README.md`
Architecture diagram	`docs/architecture.md` (§2 of this document)
ER diagram	`docs/er-diagram.md` (§3 of this document)
API documentation	`docs/api.md` (§6 of this document)
Design decisions document	`docs/design-decisions.md` (§10 of this document)
Automated tests	`packages/*/src/**/*.test.ts`, run via `npm test` (§9 of this document)